Open-Source Local VLMs in Visuo-Spatial-Temporal Memory

An Evaluation of RAVEN using Local Multimodal Embedders and VLMs on DARPA TIAMAT Simulation Data

Edison Zhu

Adviser: Professor Dhruv Shah

Graduate Mentor: Yixun Hu

Bachelor of Science in Engineering

Department of Electrical and Computer Engineering

Princeton University

April 2026

I hereby declare that I am the sole author of this thesis.

I authorize Princeton University to lend this thesis to other institutions or individuals for the purpose of scholarly research.

_____

Edison Zhu

I further authorize Princeton University to reproduce this thesis by photocopying or by other means, in total or in part, at the request of other institutions or individuals for the purpose of scholarly research.

_____

Edison Zhu

Open-Source Local VLMs in Visuo-Spatial-Temporal Memory: An Evaluation of RAVEN using Local Multimodal Embedders and VLMs on DARPA TIAMAT Simulation Data

Edison Zhu

Abstract. A robot deployed in an unfamiliar environment for hours or days has to remember what it has seen well enough to answer questions later, e.g. "where is the purple chair?" or "where did I start?". Storing every frame is memory-prohibitive; storing labels from a fixed vocabulary loses anything outside that vocabulary; storing captions loses anything the captioner doesn't bother to mention. RAVEN (Hu et al., RSS 2026) is a recent system that takes a middle path: it embeds each video frame as a high-dimensional vector alongside the robot's pose and timestamp, then at query time hands a vision–language model (VLM) three retrieval tools (text, time, and position lookups) and lets the VLM iterate until it has enough evidence to commit to an answer. The published RAVEN paper benchmarks mostly closed-source VLMs (Gemini, GPT) and reports a striking negative result in its §5.3: when the reasoning backbone is swapped for a medium-sized *open* VLM, the whole pipeline can drop *below* a baseline that just does top-1 cosine retrieval on the user's query embedding — that is, the open VLM hurts more than it helps.

This independent work asks two related questions: *how well do open-source VLMs perform inside RAVEN's tool-use loop, and what is the open-VLM penalty paying for when they fall short?* I integrate four locally-served open backbones (Gemma 3 27B, Gemma 4 31B, Qwen2.5-VL 32B, Qwen3-VL 32B) into the RAVEN harness on DARPA TIAMAT simulation data, author a 19-question evaluation suite stratified across five capability levels (direct lookup, attribute mapping, spatial reasoning, multi-step inference, and negation), and grade each answer manually. The headline result is that adding the VLM agent on top of QQMM-embed-v2 lifts **retrieval-level** accuracy from 7/19 (embedder-only baseline) to 15/19 with Gemma 4 31B — a **+42 percentage-point** jump entirely concentrated on the spatial-reasoning, multi-step, and negation levels (L3–L5), where retrieval alone is structurally insufficient. Both numbers ask the same question (*was the correct frame in the top-K retrieval?*) and so are directly comparable. Memory footprint and accuracy do not correlate across the four backbones: all four sit in a tight ≈16–22 GB VRAM band yet span 12–15 / 19 in accuracy, and the smallest of the four (Gemma 3 27B at ≈16 GB) is two questions behind the highest-scoring (Gemma 4 31B). I localize the remaining losses to *agent-side* mechanisms — retrieval loops, missing coordinate plumbing in the answer-formulation path,

an under-used non-text retriever, and a question-type classifier that sometimes routes a positional question into the prose path — rather than retrieval-substrate or base-VLM failures, and identify three framework-level fixes (query diversification, coordinate fallback, tool-use priming) that should close most of the gap to closed-VLM performance without retraining the VLM.

Acknowledgments

I thank my adviser, Professor Dhruv Shah, for proposing this project and for shaping the experimental scope across the semester. I thank Yixun Hu, the PhD student who served as my day-to-day mentor, for walking me through the RAVEN codebase, for many discussions on the design of the evaluation suite, and for honest critique of early failure-mode write-ups. Yixun is also a co-author on RAVEN, and much of the framing in this report is owed to his guidance. I thank the rest of the RAVEN authors—Zhicheng Zheng, Lihan Zha, Chunwei Xing, Rajdeep Singh, Omar Hossain, and Antonio Loquercio—for open-sourcing the reference implementation that this work builds on, and for documenting the open-VLM gap that motivated this evaluation. I thank the Princeton ECE Department for compute resources (2× NVIDIA L40 via the departmental cluster) used to run the open-VLM backbones locally. Any errors in interpretation are my own.

Contents

1. Introduction

1.1 Problem statement and scope

A robot deployed for hours or days in an unfamiliar environment must remember what it has seen in a way that is **compact** (memory cannot scale linearly with the number of frames captured), **grounded in space and time** (downstream queries are typically about *where* and *when*), and **information-rich** (open-vocabulary visual detail must survive into retrieval, not be discarded at storage time). The two dominant families of long-horizon memory both struggle under at least one of these constraints. *Explicit semantic maps* (point clouds or grids tagged with discrete labels from a fixed vocabulary) are compact and grounded, but lose any concept that was not pre-registered: a "red Tesla in the driveway" cannot be recovered if the vocabulary only knows "car." *Captioning-based pipelines* (most prominently ReMEmbR [2] and its descendants) convert frames into natural-language captions before embedding, so they are open-vocabulary and grounded, but the image-to-text step introduces a **captioning bottleneck**: textures, layouts, and fine spatial details that cannot be concisely verbalized are lost before retrieval ever begins. A third line, long-context VLMs reading every frame at query time, is information-preserving but does not scale: context windows of even the largest models are exhausted at hundreds of frames, far below the trajectory lengths a deployed robot accumulates.

RAVEN (*RetrievAl via Visual Embeddings for Navigation*; Hu et al. [1]) bypasses the captioning step entirely. Raw RGB frames are passed through a multimodal embedder (in the reference implementation, QQMM-embed-v2 [6]); each embedding z_i is indexed into a vector database together with the robot's pose $p_i = (x, y, z)$, orientation θ_i, timestamp t_i, and a path to the on-disk PNG. The frames themselves are not copied into the database — only the path is stored, and the agent loads the image lazily from disk when a retrieval brings it into the working set. At query time, a VLM agent iteratively invokes three retrieval primitives (text-based, time-based, and position-based) until accumulated evidence is sufficient to emit an answer. The result is a memory that is sub-linear in retrieval cost, open-vocabulary by construction, and order-of-magnitude smaller than storing the frames themselves.

The RAVEN paper benchmarks primarily *closed* VLMs (Gemini-2.5-Flash, Gemini-3-Pro, GPT-5.2) on three datasets and reports an important negative result in §5.3: when the reasoning backbone is swapped for a medium-sized *open* VLM (Gemma3-27B, Qwen3-VL-32B), performance on Habitat-simulation queries can drop *below* the embedder-only floor, i.e., below the performance achievable by stripping out the agent and just doing top-1 cosine retrieval on the user's query embedding. The paper recommends the embedder-only pipeline

for offline deployments under such conditions—a pattern this work partially disconfirms in §4.2, where all four open backbones beat the embedder-only floor.

This observation is the starting point of the present work. For a researcher who needs to deploy on-device (DARPA-style, air-gapped, no cloud calls), the choice between closed and open VLMs is not free. **What, exactly, is the open-VLM penalty paying for?** Is the loss in (a) the retrieval substrate transferring poorly to new simulation data, (b) the VLM's language understanding, (c) the VLM's tool-use strategy, or (d) framework-level plumbing between the retrieval result and the final coordinate answer? Each of these has a different fix, a different cost, and a different prognosis for offline deployment. The contribution of this independent work is to localize the loss.

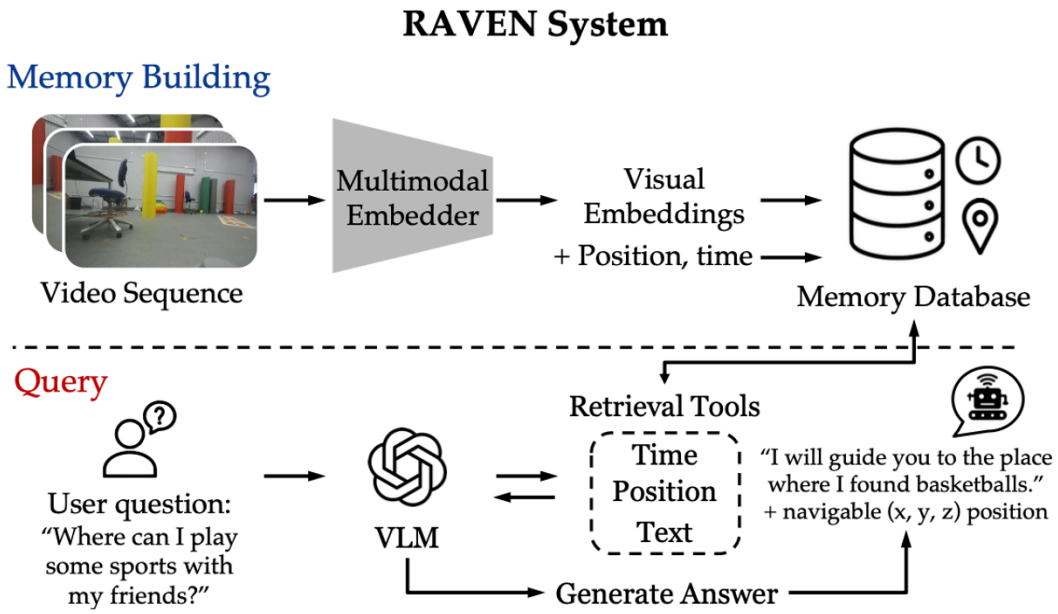

Figure 1. The RAVEN system. At ingest time (top), the multimodal embedder converts each video frame into a visual embedding stored in a vector database alongside the robot's pose and timestamp. At query time (bottom), a vision–language model iteratively invokes three retrieval tools (text, time, position) over the memory database, accumulates evidence, and emits an answer with a navigable (x, y, z) position. Reproduced from [1].

1.2 Contributions

This independent work makes three scoped contributions, each addressing a specific gap in the RAVEN open-VLM coverage:

1. **Open-VLM extension of RAVEN on DARPA TIAMAT simulation data.** I integrate four locally-served open VLMs (Gemma 3 27B, Gemma 4 31B, Qwen2.5-VL

32B, and Qwen3-VL 32B) into the RAVEN harness via Ollama, add per-model YAML configurations to the framework, and run each against a Habitat smoke trajectory drawn from the DARPA TIAMAT competition stack.

2. **A difficulty-stratified evaluation suite (L1–L5).** I author a nineteen-question suite over the smoke trajectory in which each question is tagged with a capability level: direct object recall (L1), indirect attribute mapping (L2), spatial reasoning (L3), multi-step inference (L4), and negation/disambiguation (L5). Each level is designed to *isolate* a specific RAVEN capability: L4_3 ("where did the agent start its exploration?") is unanswerable by text retrieval alone and forces the time-based retriever; L5_1 ("find a chair that is NOT in the kitchen") forces post-retrieval filtering. Ground truth is multi-position, allowing the evaluator to distinguish "right room, wrong pose" from "wrong room entirely."

3. **Failure-mode diagnosis.** I produce a qualitative readout that identifies the dominant agent-side failure patterns—most prominently *retrieval loops* in which the VLM re-issues the identical query four to six times rather than diversifying—and propose three framework-level fixes (query diversification, coordinate fallback, tool-use priming) that do not require retraining the VLM and do not touch the retrieval substrate.

This is a **replication-and-extension study**, not a novel method. The retrieval substrate, the tool-use loop, and the embedder-only baseline are all from the RAVEN paper. The contribution here is open-VLM coverage on new evaluation data, the difficulty-stratified suite, and the failure-mode taxonomy.

2. Background and Related Work

2.1 Memory representations for embodied agents

The literature on long-horizon memory for embodied agents has converged on three approaches, each with a known failure mode.

Closed-vocabulary semantic maps. Classical SLAM produces a metric map; semantic SLAM tags map elements with discrete object labels drawn from a fixed taxonomy. The advantage is compactness and consistency with downstream planners. The disadvantage is hard: out-of-vocabulary objects cannot be retrieved at all, and retraining the perception stack to add a class is costly.

Captioning-based open-vocabulary pipelines. ReMEmbR [2] and its descendants embed natural-language captions of frames rather than the frames themselves. This solves

the closed-vocabulary problem but introduces the *captioning bottleneck*: anything the captioner does not say cannot be retrieved. Fine spatial layout, subtle textures, and unusual object configurations are systematically lost. The RAVEN paper's NaVQA experiments show ReMEmbR underperforms direct visual embedding by 20–30% on hard queries.

Long-context VLMs. Reading every frame at query time preserves all visual information but does not scale. Context window limitations of even the largest VLMs are exceeded at trajectory lengths of a few hundred to a few thousand frames; cost scales linearly per query.

Recurrent / RNN-based memories. Forget over long horizons in a way that is hard to control or interrogate.

2.2 The RAVEN system

RAVEN proposes two design shifts that together address the limitations above. First, **embeddings, not captions**: modern multimodal encoders (CLIP [4], SigLIP [5], QQMM-embed-v2 [6]) are sufficiently aligned with language that raw visual embeddings can be retrieved with text queries, skipping the lossy image-to-text step. Second, **retrieval, not attention**: instead of passing every stored frame to a long-context VLM, the agent invokes targeted retrieval over a vector database and operates on a *working memory* of retrieved frames, achieving sub-linear retrieval complexity and order-of-magnitude lower per-query memory pressure.

The RAVEN agent loop maintains a context R_τ of accumulated retrieved frames, parses the user query, and at each step either emits a final answer or issues a retrieval call. Three retrieval primitives are exposed:

- **Text-based.** The VLM proposes a query string $\hat{q}$. The system computes $\hat{z}_q = f_{\text{enc}}(\hat{q})$ and returns the top-K frames by cosine distance. This is the workhorse primitive.
- **Time-based.** Returns K *consecutive* frames anchored at a queried timestamp. The RAVEN paper notes this performs better than nearest-neighbor retrieval in time, because consecutive frames are more diagnostic of trajectory state.
- **Position-based.** Returns K spatial nearest neighbors to a VLM-proposed coordinate $(\hat{x}, \hat{y}, \hat{z})$.

The reported results in the RAVEN paper are strong on the closed-VLM track. RAVEN with QQMM-v2 outperforms ReMEmbR by up to 30% on hard queries; quality is maintained as memory scales to $\sim$3,000 frames on the FindingDory long subset (where a VLM-only baseline degrades sharply); and $\geq$ 97% success is achieved on real Unitree Go1 rollouts. The

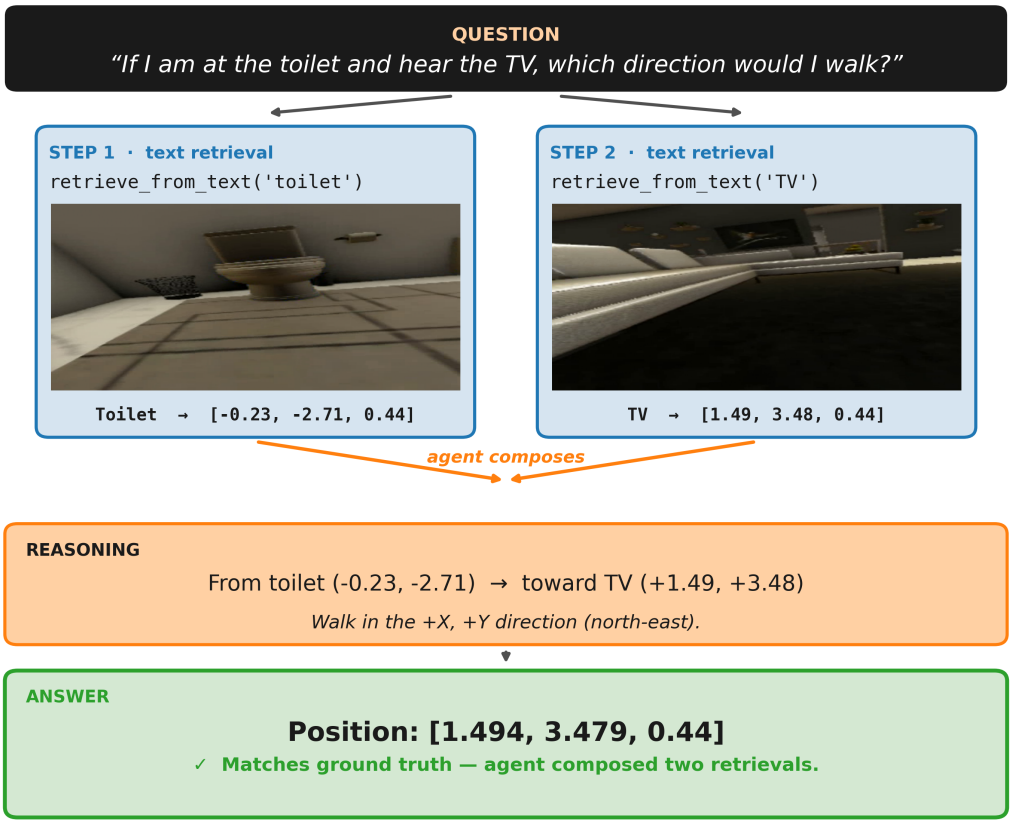

Figure 2. RAVEN's tool-use loop. The VLM parses the query, decides whether to emit an answer or call a retrieval primitive, accumulates evidence in R_τ, and repeats. The three retrieval primitives are exposed as tools.

open-VLM concession in §5.3, however, is the relevant data point for offline deployment: medium-sized open VLMs (Gemma3-27B, Qwen3-VL-32B) can drop below embedder-only retrieval on Habitat-simulation queries, particularly on tasks requiring multi-step composition.

2.3 Where this work fits in the literature

Three nearby lines of recent work each address adjacent questions but not the one this thesis pursues:

- **Tool-use evaluations of open VLMs.** Recent work has measured how reliably open VLMs invoke tools in general agentic settings. These benchmarks are not embodied; they do not stress the visual retrieval substrate.
- **Retrieval-augmented generation (RAG) for VLMs.** Adjacent in mechanism

but the retrieved corpus is text/document, not visuo-spatial-temporal robot memory. Failure modes do not transfer.

- **Habitat / FindingDory [3] benchmark papers.** Provide standardized question suites over indoor scenes, but the published RAVEN evaluation does not break out per-difficulty performance for open VLMs at the granularity needed to localize failure.

This work sits in the gap precisely identified by the RAVEN authors in §5.3: *if open-VLM RAVEN is already at or below the embedder-only floor, what is actually driving the loss?* The L1–L5 evaluation suite is the instrument designed to answer this.

3. Methods

3.1 Environment and base system

I evaluate inside the **DARPA TIAMAT** competition stack, using the `remembr_static_eval_vlm.py` harness and the `darpa_vlm_prompts` prompt set. All code is a fork of the RAVEN authors' reference implementation; **no retrieval, tool, or prompt changes are introduced**. The only modifications to the framework are (i) per-model YAML config additions for the four new VLM backbones, (ii) the question file specifying the L1–L5 suite, and (iii) the evaluator's grading script. This scope discipline is intentional: any performance change must be attributable to the swapped VLM, not to incidental framework drift.

3.2 Memory construction

The **smoke_v2** trajectory comprises 843 egocentric Habitat frames captured over approximately 14 minutes of simulated walking through an indoor residential scene (entry/hallway, living room, kitchen, bathroom, dining transition area). Per-frame poses are stored in `position_data_habitat_smoke_v2.csv` as (x, y, z, yaw) in scene coordinates.

Each frame is embedded with **QQMM-embed-v2** (3584-dim; Hugging Face: `youzexue/QQMM-embed-v`) and indexed into FAISS [7] alongside the position p_i, orientation θ_i, timestamp t_i, and the path to the on-disk PNG, following the RAVEN `VLMMemoryItem` schema. The image bytes themselves are not duplicated into the index; the path is loaded lazily when the agent retrieves the entry. Top-K is fixed at 5 throughout, using cosine distance. This matches the RAVEN paper's defaults; varying K is out of scope.

3.3 Reasoning backbones

I integrate four open-source VLMs (Gemma 3 27B, Gemma 4 31B, Qwen2.5-VL 32B, and Qwen3-VL 32B) by adding YAML configurations that reference Ollama-served model tags. The harness instantiates each as the reasoning LLM inside the RAVEN tool-use loop. Hardware is two NVIDIA L40 GPUs (each 48 GB VRAM): the Ollama VLM is pinned to GPU 0; the QQMM embedder runs on GPU 1.

The headline result reported throughout this thesis uses **Gemma 4 31B + QQMM-embed-v2**, which is referred to as the "RAVEN agent" configuration. This is the strongest open configuration that fit fully on the available L40 hardware and is closest in scale to the medium-open backbones the RAVEN paper concedes are difficult.

Memory footprint of the open VLM backbones. Every backbone in this study is loaded as a 4-bit quantized build; the unquantized BF16 weights of a 32B-parameter VLM are roughly 64 GB and do not fit on a single L40 at all. Table 1 reports the approximate VRAM footprint for each backbone, sourced from published model cards and standard GGUF/Ollama load sizes for the relevant 4-bit quantization (Q4_0 or q4_K_M). These are not `nvidia-smi` measurements taken in this work; they are representative numbers from the model providers and community deployments, and they are reported here only to show that all four backbones fit comfortably on a 48 GB L40 with substantial KV-cache headroom. The footprints cluster in a tight 16–22 GB band despite the models spanning 27–32B parameters, because 4-bit quantization compresses weights into roughly the same per-parameter footprint regardless of family. Gemma 3 27B is the smallest at ≈16 GB; Gemma 4 31B and Qwen3-VL 32B both sit near 20 GB; Qwen2.5-VL 32B is intermediate at ≈19 GB. The footprint differences are small enough that "which model fits" is *not* the operative constraint, and the performance differences reported below cannot be charged to memory-headroom artifacts.

Table 1. Approximate VRAM footprint of the four open VLM backbones when served via Ollama in 4-bit quantized form. Numbers are sourced from each model's published card and from community-reported load sizes for the relevant quantization; they are not measured at runtime in this work. All four fit comfortably on a single 48 GB L40 with substantial KV-cache headroom for the tool-use trace.

Model (Ollama tag)	Quantization	Approx. VRAM
`gemma3:27b`	q4_K_M	≈16 GB
`gemma4:31b`	Q4_0	≈20 GB
`qwen2.5vl:32b-q4_K_M`	q4_K_M	≈19 GB
`qwen3-vl:32b-instruct-q4_K_M`	q4_K_M	≈20–22 GB

3.4 Evaluation suite (L1–L5)

I author a nineteen-question, five-level suite over the smoke_v2 scene specifically designed to isolate the pipeline capabilities the rest of this thesis tries to localize.

Why stratify by capability. The thesis is asking *where along the difficulty curve does the LLM stop being optional?*, and a single aggregate accuracy number cannot answer that. Two questions that look syntactically similar can exercise completely different parts of the pipeline: "Where is the purple chair?" is solved by top-1 keyword retrieval; "Find a chair that is NOT in the kitchen" requires the agent to retrieve *and then filter*. Pooling them obscures exactly the embedder floor vs. agent ceiling divergence that the rest of §4 sets out to chart. I therefore stratify the suite by structural capability rather than surface form, so per-level scores read as *which capabilities are present* rather than as a noisy aggregate.

Design constraints on the level ordering. The L1→L5 progression is therefore organized by the structural capability each level demands of the pipeline, not by perceived difficulty: L1 demands only keyword retrieval; L2 adds a concept→object mapping that the embedder can sometimes handle and the VLM almost always can; L3 demands relational reasoning over multiple retrieved frames; L4 demands composition, often involving the time or position retriever rather than text alone; L5 demands filtering or exclusion over a retrieval set, which neither the embedder nor a single forward pass can produce. Each level is constructed so that a system passing Lk must in principle have all the capabilities required by L(k−1)—a monotone-capability ordering, not a difficulty-rating curve. This makes the per-level breakdown in §4.3 interpretable as a *capability ladder*: the level at which an embedder-only or agent configuration first stops scoring is a direct readout of the missing capability, not a noisy aggregate.

Adversarial questions. Three of the nineteen questions, all at L4–L5, are *adversarial* to a single RAVEN component and designed to fail unless that component is exercised correctly: L4_3 ("Where did the agent start?") cannot be solved by text retrieval and forces the time retriever; L5_4 ("Where did the agent spend the most time?") forces temporal-statistical reasoning over the trajectory rather than per-frame retrieval; L5_1 ("a chair that is NOT in the kitchen") forces post-retrieval filtering, which a confident top-1 retrieval will get wrong. L1–L3 contain no adversarial questions by design, because those levels are meant to measure the joint embedder+VLM ceiling on lookup-style queries rather than to stress specific framework components. The three adversarial cases are what let the failure-mode

diagnosis in §5 attribute losses to specific framework components rather than to "the agent in general."

Table 2. The five-level evaluation taxonomy. Each level isolates a capability; three questions across L4–L5 are specifically adversarial to a single RAVEN component (L4_3, L5_1, L5_4). L1–L3 contain no adversarial questions by design.

Level	Name	Capability probed	Example
L1	Direct	Object recall from keyword	Where is the purple chair?
L2	Indirect	Concept → object mapping	Where can I sit down comfortably?
L3	Spatial	Relational reasoning over scene	What room is closest to the entry?
L4	Multi-step	Composed inference (often temporal)	Where did the agent start its exploration?
L5	Negation	Filter / exclusion over retrieval	Find a chair that is NOT in the kitchen.

L1, L3, L4, and L5 contain four questions each; L2 contains three, for a total of nineteen scoreable questions. Ground truth is **multi-position**: a question may have two or three acceptable (x, y, z) answers, which lets the grading scheme of §3.5 distinguish "right room, wrong exact coordinate" from "wrong room entirely." The full text of all nineteen questions, with capability tags and ground-truth positions, appears in Appendix A.

3.5 Grading scheme

Each answer is graded manually against a two-path rubric. An answer is **correct** if *either* path passes:

1. Given a predicted position $\hat{p}$ and a ground-truth set $\{p_j^*\}$, define $d = \min_j \|\hat{p} - p_j^*\|_2$. The first path passes if $d \leq 2\,\text{m}$. The 2-meter threshold corresponds to "same furniture group" in the test scene.

2. When the agent returns no coordinate, returns a coordinate that fails path 1, or answers a non-positional question (binary, text), I manually inspect the retrieved top-K frames and the model's response text. The second path passes if the retrieved frames visually show the queried object or room *and* the response correctly identifies it. For non-positional questions, this is the only available path: the response text must be correct on its merits given the retrieved evidence.

An answer is **wrong** if both paths fail (i.e., the coordinate is more than 2 m from any ground-truth pose *and* manual visual inspection of the retrieved frames does not support the answer, or the reasoning text is incorrect for non-positional questions).

This rubric is what allows "right room, wrong pose" to be graded distinctly from "wrong room entirely" (§3.4). A correct prose answer with no emitted coordinate (e.g., "the entryway") passes the rubric whenever the retrieved frames support it. All headline accuracy numbers in this work use this two-path rubric.

3.6 Baselines

Two baselines, both established by the RAVEN paper:

- **Embedder-only.** Four contrastive image–text encoders are evaluated with no LLM in the loop: top-1 retrieval on the user query embedding, with the answer coordinate taken as the centroid of the retrieved frame's pose. The four models are CLIP ViT-L-14 [4] (768-dim), CLIP ViT-H-14 (1024-dim), SigLIP-384 [5] (1152-dim), and QQMM-embed-v2 [6] (3584-dim).
- **ReMEmbR** [2]. Caption-based baseline. Quantitative ReMEmbR numbers are inherited from the RAVEN paper's reported patterns; this work does not re-run the captioner end-to-end on smoke_v2.

4. Experiments and Results

4.1 Embedder-only baseline

Mean top-1 retrieval similarity across the nineteen queries for the four contrastive embedders evaluated:

Model	Dim	Mean similarity
SigLIP-384 (ViT-SO400M-14)	1152	0.108
CLIP ViT-L-14 (OpenAI)	768	0.205
CLIP ViT-H-14 (LAION-2B)	1024	0.249
QQMM-embed-v2	**3584**	**0.356**

Observation 4.1. QQMM-embed-v2 achieves substantially higher absolute similarity than CLIP and SigLIP on the DARPA simulation data—consistent with its navigation-specialized training distribution. Among the two general-purpose contrastive embedders, CLIP achieves approximately twice SigLIP's absolute similarity, *inverting* the relative ordering reported on some web benchmarks. This is consistent with a sim-to-real gap: SigLIP's training distribution is heavier on natural photographs, while contrastive CLIP generalizes

more evenly to the rendered Habitat imagery. Within the CLIP-style contrastive family, similarity rises monotonically with embedding dimensionality (ViT-L-14 at 768-dim → ViT-H-14 at 1024-dim → QQMM-embed-v2 at 3584-dim; 0.205 → 0.249 → 0.356); SigLIP breaks this trend, suggesting the dimensionality effect is a within-family capacity signal rather than a general law—training distribution dominates across families.

Observation 4.2. Absolute similarities span ≈0.11–0.36, with three of the four embedders sitting below the ∼0.3 typical on real-world benchmarks—a quantitative fingerprint of the sim-to-real distributional shift that any deployment on Habitat-rendered data must contend with.

4.2 RAVEN with open VLM backbones

I ran the full RAVEN agent against the 19-question suite with each of the four open backbones, all on the same QQMM-embed-v2 retriever, and graded each answer manually at the retrieval level (was the correct frame in the top-K retrieval, regardless of how the VLM formatted its final answer?):

Model	Approx. VRAM	Correct	Wrong
Gemma 4 31B	**≈20 GB**	**15 / 19**	**4 / 19**
Gemma 3 27B	≈16 GB	13 / 19	6 / 19
Qwen2.5-VL 32B	≈19 GB	13 / 19	6 / 19
Qwen3-VL 32B	≈20–22 GB	12 / 19	7 / 19

Observation 4.3. All four backbones land in a tight 16–22 GB VRAM band, so any performance differences cannot be attributed to memory headroom or which model fits. Gemma 4 31B retrieves the correct frame on 15 of 19 questions; the other three open VLMs cluster at 12–13 / 19. Notably, Gemma 3 27B is the *smallest* of the four at ≈16 GB and still scores 13/19, two questions behind the headline run. Memory footprint and retrieval-level accuracy do not correlate on this suite (see Figure 3).

Observation 4.4. The bottleneck is downstream of retrieval, in answer formulation, and it shows up across *every* backbone I tried. The failure modes catalogued in §5 recur in qualitatively similar form across all four VLMs. The per-level breakdown in Figure 4 shows all four agents alongside the embedder baseline; latency, tool-call counts, frame-reuse, and the walked-through reasoning traces in the remainder of §4 are reported for the headline Gemma 4 31B run only.

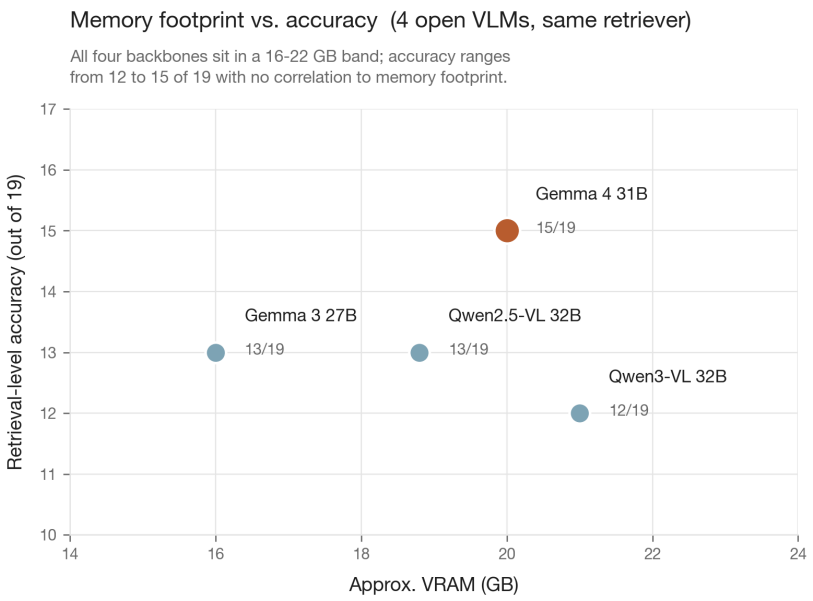

Figure 3. Approximate VRAM vs. retrieval-level accuracy on the 19-question suite, for the four open VLM backbones I evaluated. VRAM numbers are sourced from published model cards and standard GGUF/Ollama load sizes (not measured at runtime in this work). All four backbones cluster in a 16–22 GB band and accuracy ranges from 12–15 / 19, with no clear correlation: the smallest backbone (Gemma 3 27B, ≈16 GB) is mid-pack at 13/19, and the headline winner (Gemma 4 31B, ≈20 GB) is not the smallest. Performance differences on this suite cannot be charged to memory-headroom or model-fit artifacts.

4.3 Per-level breakdown

The single most informative chart in this thesis is the per-level comparison of QQMM (embedder-only) against each of the four full RAVEN agents, reproduced in Figure 4.

Qualitative agent behavior, level by level (Gemma 4 31B + QQMM):

L1 (direct). Near-ceiling. All four objects are retrieved in top-3 and the model emits coordinates from the retrieved frame's metadata cleanly. The embedder-only pipeline also succeeds on L1. *The VLM adds essentially nothing here*; the scaling argument therefore must come from L3–L5.

L2 (indirect/attribute). Strong. The VLM translates "watch the news" → `"television"`, "sit down comfortably" → `"comfortable seating"`, "wash my hands" → `"sink"`. Concept-to-object generalization via QQMM survives the sim-to-real gap.

L3 (spatial). Mixed in raw output, strong in retrieval. The agent produces correct *prose* answers ("the entryway," "the living room") in three of four L3 questions, but frequently omits coordinates in the answer payload—a framework plumbing issue diagnosed in §5.

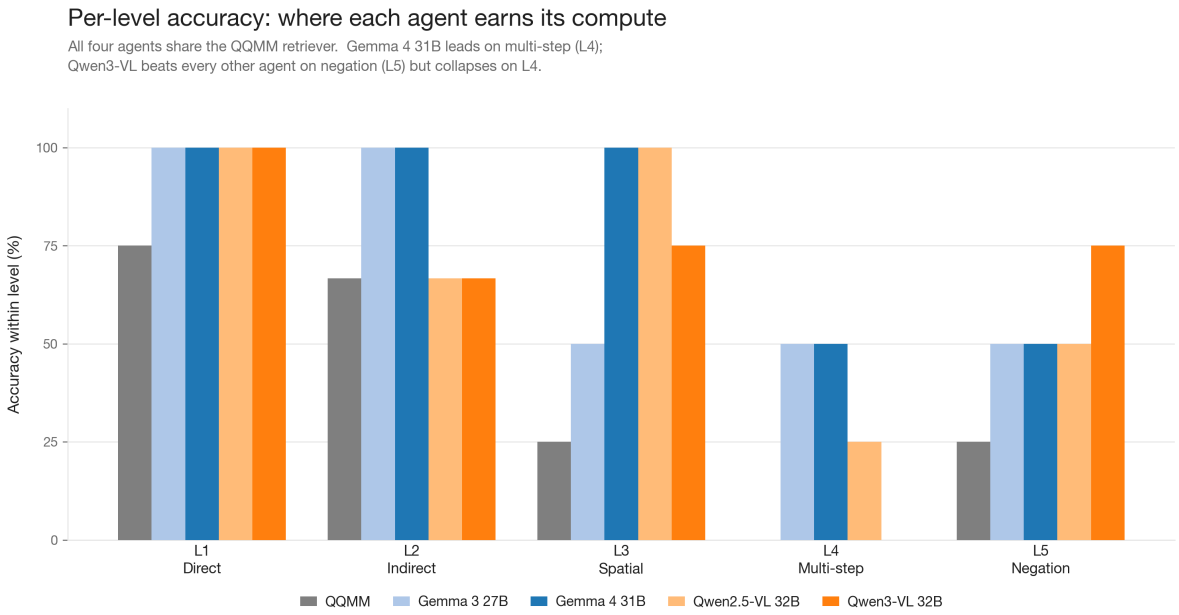

Figure 4. Per-level accuracy across all four RAVEN agents plus the QQMM embedder-only baseline. All four agents share the same QQMM retriever, so within-level differences isolate the contribution of the VLM. The agents are tied with the embedder at L1 and L2 (the easy half), but each opens a sustained gap on L3–L5. Gemma 4 31B leads on multi-step (L4); Qwen3-VL is the only agent above 50% on negation (L5) but collapses on L4.

L4 (multi-step). The standout result is L4_3—"Where did the agent start its exploration?"—the only query in the suite where the model spontaneously pivoted off text retrieval. After `"start of exploration"` returned generic frames, it called `retrieve_from_time("00:00:00")` and landed on `frame_000000`.

L5 (negation). The model reformulates rather than filters: "flat surface that is NOT a floor" → `"a flat surface like a table or a counter"`. The agent gets two of four on L5; embedder-only gets one of four (a lucky retrieval on L5_1, where top-1 cosine on "chair" lands directly on the purple-chair frame, which happens to satisfy the negation).

Walked-through reasoning traces

The per-level summary above is the macro view; the texture of why the agent succeeds or fails is in the individual traces. I walk through three representative cases below.

L4_2, "Where would I find something to clean a spill?" (✓). This is the cleanest demonstration of *concept-to-object* retrieval that the embedder alone cannot do. The agent

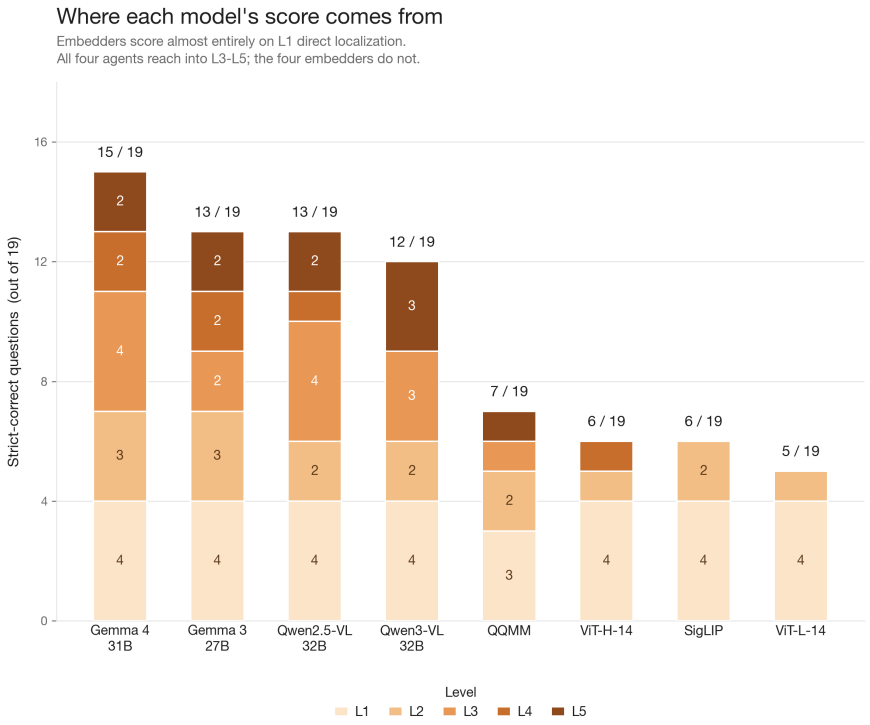

Figure 5. Where each model's score comes from. One narrow vertical column per model (four VLM agents and four embedder baselines), segmented by level (L1 light, L5 dark). Embedders score almost entirely on the L1 segment and bottom out from L3 onward; the VLM agents are the only configurations that reach L4 with any consistency.

issued a single text retrieval with the query `"cleaning supplies"` and the QQMM retriever surfaced `frame_000226` (a mop) as the top-2 result. The VLM then emitted the mop's pose as the answer. Embedder-only retrieval on the user's literal query string "where would I find something to clean a spill" lands on irrelevant kitchen-floor frames; the VLM's reformulation to "cleaning supplies" is what makes the retrieval succeed. This is the L4 win that the embedder cannot replicate and is consistent with the per-level gap in Figure 4.

L4_3, "Where did the agent start its exploration?" (✓). The standout trace. The agent first issued `"start of exploration"` (a literal paraphrase of the question) and got back generic indoor frames—no temporal anchor. Recognizing the failure, it switched tools: it called `retrieve_from_time("00:00:00")`, which returns frames anchored at trajectory start. The result was `frame_000000` itself, and the agent emitted that frame's pose as the answer. This is the only query in the entire suite where a non-text retriever was spontaneously invoked. It is positive evidence that open VLMs *can* orchestrate multi-tool strategies inside RAVEN; the rarity of this behavior (§5, item 5.5) is then the evidence that they don't reach for it without prompting.

L5_3, "Find a flat surface that is NOT a floor" (×). The instructive failure. The agent reformulated the negation rather than filtering it: it issued `"a flat surface like a table or a counter"` as the text query. QQMM returned `frame_000343`, which the VLM correctly identified as containing marble texture. However, it then mislabeled the surface as a "marble countertop" when it was in fact a marble *floor*. The retrieval was correct in the sense of returning a visually relevant frame; the *interpretation* of the surface type was wrong. This is the texture–surface confusion mode of §5.4: the embedder cannot, in general, distinguish horizontal floor from horizontal countertop because it produces a "flat marble surface" embedding either way. As §5.4 argues, the failure originates at the embedder but a single VLM-level disambiguation call ("floor or countertop?") would resolve it without retraining the retriever. Reformulation around negation also matters here: an alternate strategy would have been to retrieve all flat-surface candidates, *filter out the floors*, and then choose; the agent did not adopt this strategy.

L4_4, "If I'm at the toilet and hear the TV, which direction would I walk?" (×). A different failure mode: retrieval succeeds on the bathroom-side endpoint but the geometric reasoning step is missing. The agent retrieved the toilet (`frame_000628`) and described the living room with the TV in prose, but it never produced a directional vector or even a unit-bearing answer. The output was a description of the TV's location, not a direction. This is consistent with L4_4 being the hardest L4 question in the suite: it requires the VLM to do a small geometric computation that text retrieval alone cannot supply, and the prompt does not prime the model to do it.

4.4 Embedding dimension does not crack the ceiling

A natural hypothesis when an embedder underperforms is "a bigger embedder would fix it." Figure 6 tests this directly and Table 3 reports the underlying counts: across a 4.7× range of embedding dimensions (768 → 3584), retrieval-level accuracy creeps up only from 5/19 to 7/19 (26% to 37%). Adding the VLM agent on the *same* 3584-dim QQMM retriever more than doubles accuracy to 15/19 (79%) in a single step. The retrieval substrate is not the binding constraint. Note that the comparison gives the agent two affordances the embedder baseline does not have—top-K (K=5) and within-loop query reformulation—so the +8-question lift bounds the agent's marginal contribution from above and is partly attributable to those affordances rather than to the VLM's reasoning alone (§6.4).

Table 3. Retrieval-level accuracy (≤ 2 m or visual-path pass) by retriever embedding dimension. All embedder-only rows use top-1 retrieval with no LLM in the loop; the final row swaps in the full RAVEN agent (top-K=5 plus reformulation) on top of the same QQMM retriever. The 4.7× dimension sweep moves only two questions; adding the agent moves eight.

Configuration	Embedding dim	Correct
CLIP ViT-L-14 (OpenAI)	768	5 / 19 (26%)
CLIP ViT-H-14 (LAION-2B)	1024	6 / 19 (32%)
SigLIP-384 (ViT-SO400M-14)	1152	6 / 19 (32%)
QQMM-embed-v2	3584	7 / 19 (37%)
Gemma 4 31B + QQMM (agent)	**3584**	**15 / 19 (79%)**

4.5 Frame-reuse analysis

A striking qualitative finding in the Gemma 4 31B traces is that the model consistently re-retrieves the same landmark frames for semantically related queries. Three frames each serve four distinct questions, accounting for 12 of the 19 queries between them; the remaining seven questions touch other frames once each. Table 4 enumerates the mapping.

Table 4. The three "hub" frames the **Gemma 4 31B + QQMM** agent re-uses across the suite. Each row corresponds to one canonical landmark frame in the QQMM index; the right column lists the four distinct questions for which the agent's top-K retrieval surfaced this same frame. Frame-reuse data shown here are derived from the Gemma 4 31B run only; equivalent breakdowns for the other three open backbones are not reported.

Frame	Landmark	# Questions	Questions answered via this frame
`frame_000238`	red/purple chair	4	L1_1 (purple chair), L2_1 (sit comfortably), L5_1 (chair NOT in kitchen), L5_2 (seating closest to bath)
`frame_000628`	toilet	4	L1_2 (toilet), L2_2 (wash hands), L3_2 (kitchen → bathroom), L4_4 (toilet → TV direction)
`frame_000753`	TV / living room	4	L2_3 (watch the news), L3_3 (most furniture), L3_4 (seating that faces TV), L4_4 (toilet → TV direction)

This is interpretable as the VLM maintaining an implicit *landmark map* of the scene: once QQMM localizes a canonical frame for an object concept, the agent re-indexes into it across queries rather than treating each query from scratch. Notably, the chair- and toilet-anchored frames each cover questions spanning four of the five difficulty levels (L1 through L5), which is direct evidence that re-use is concept-driven—the same physical landmark is the right answer-anchor whether the query is a direct lookup, an indirect attribute, a spatial

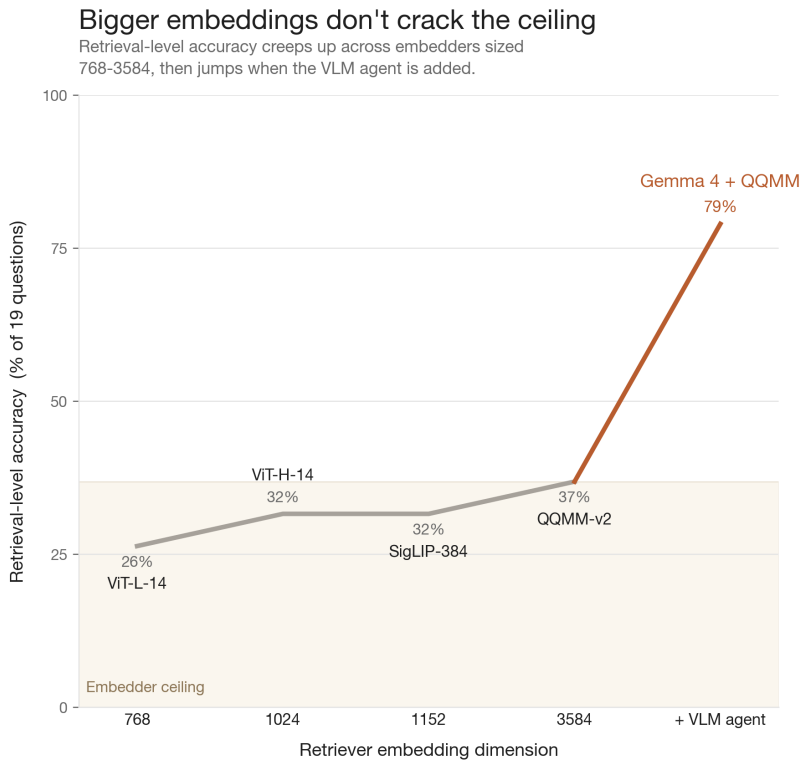

Figure 6. Retrieval-level accuracy as a function of retriever embedding dimension. The four embedder-only configurations (left of the dashed step) creep from 26% to 37% across a 4.7× dimension sweep; adding the Gemma 4 31B agent on top of the same QQMM retriever (rightmost point) jumps to 79%. The retrieval substrate is not the binding constraint.

relation, or a negation—rather than shallow surface-form matching. This is consistent with RAVEN's design intent.

4.6 Latency and tool-call counts

Two of the qualitative claims in §5—*retrieval loops* and *rare multi-tool strategy*—are quantifiable from the existing run's `debug_logs`. Parsing the Gemma 4 31B + QQMM log directly:

- **Total wall-clock for the suite:** 30.8 minutes (mean 97 s per question; median 74 s).
- **Latency rises with level:** L1 mean 67 s, L2 66 s, L3 100 s, L4 121 s, L5 127 s. L5_4 was the slowest single question at 228 s.
- **Total tool calls across 19 questions:** 112 text retrievals, 3 time retrievals, 0 position retrievals (115 calls in total).
- **All 3 time-retriever calls were on L4_3** (the agent-start question). The position retriever was never invoked across the suite, hard quantitative confirmation of the *rare multi-tool strategy* pattern in item 5.5 of §5.

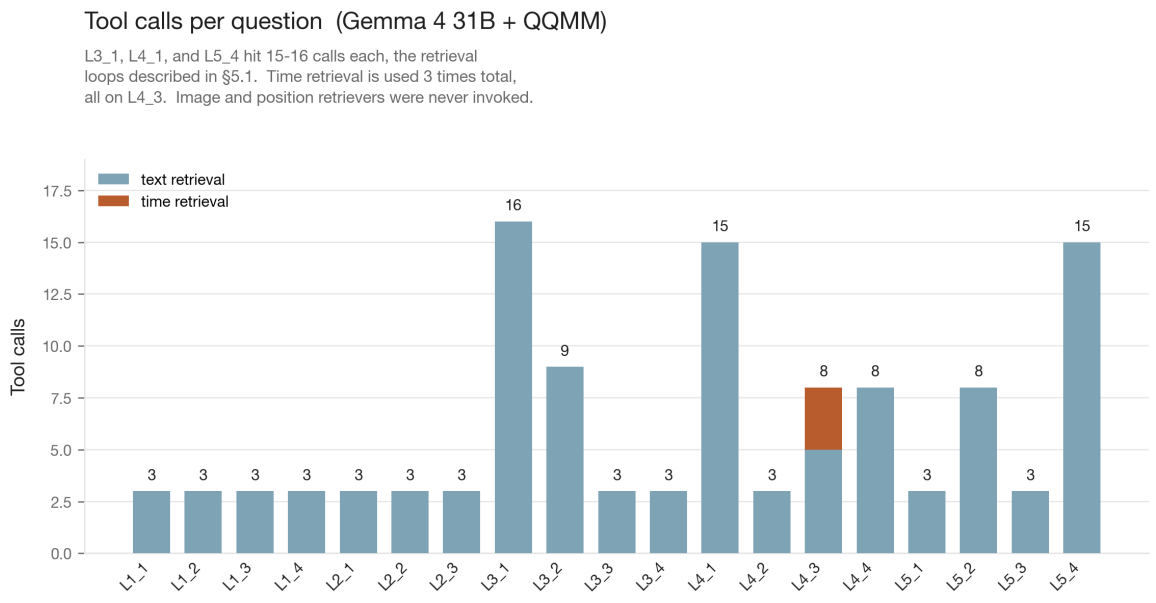

Figure 7. Tool-call count per question (Gemma 4 31B + QQMM). Most questions resolve in 3 calls (one user prompt, one tool invocation, one tool response). L3_1, L4_1, and L5_4 hit 15–16 calls each—the retrieval-loop pattern of §5.1, now quantified. Time retrieval is exclusively used on L4_3.

The retrieval-loop pattern is now visible as a number rather than an anecdote: *three* of the nineteen questions account for roughly two-fifths of the total tool-call budget (46 of 115). Each of those three (L3_1, L4_1, L5_4) re-issued the same query 5–6 times before emitting an answer, and each took ≥ 90 s of wall-clock time.

An L1/L2 question costs about a minute on this hardware; an L4/L5 question costs two. If a workload is dominated by L1/L2, the per-query overhead of running the agent at all is the target for the gating mechanism in §7.

5. Failure-Mode Diagnosis

The previous section established *that* open-VLM RAVEN tracks the embedder-only floor on L1–L2 and opens a sustained gap on L3–L5. This section asks *why* those L3–L5 results look the way they do, and where the lever for improvement sits. I localize six distinct failure modes; in §6 I show how three of them are tractable at the framework layer.

5.1 Retrieval loops dominate the failure cases. On the questions that fail, the VLM tends to re-issue the *identical* query four to six times rather than varying phrasing or switching retrieval primitives. I observed `"coffee machine"` repeated six times in one L4 trace and `"entryway"` repeated six times in another, always returning the same top-K frames. The

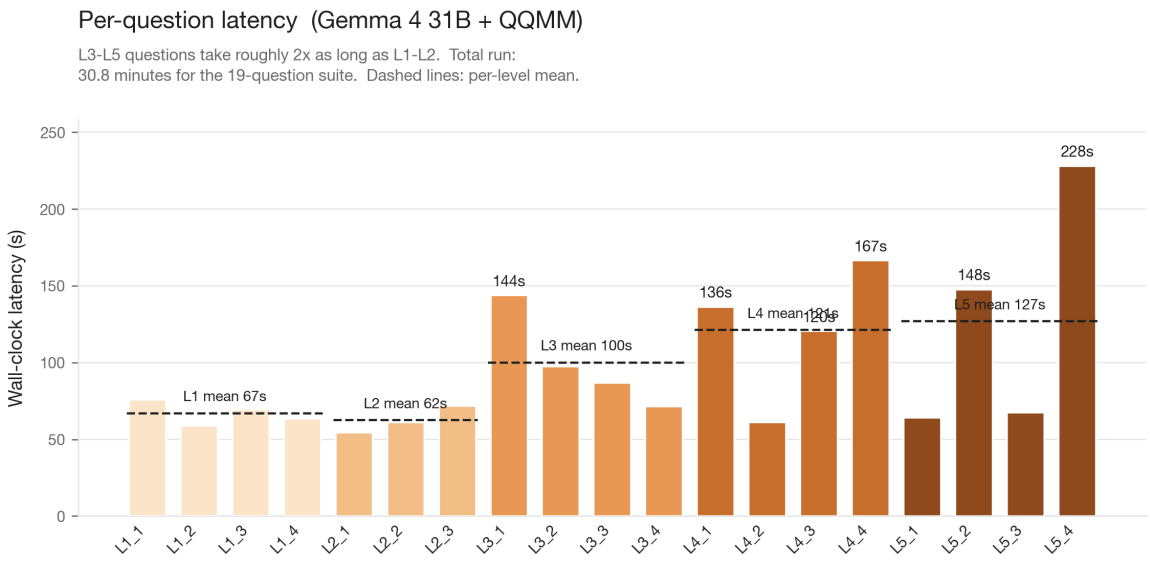

Figure 8. Per-question wall-clock latency (Gemma 4 31B + QQMM). L3–L5 questions take roughly twice as long as L1–L2. The distribution is right-tailed: median 74 s, max 228 s. Dashed lines show per-level means.

tool-use loop has no backoff, no query-diversification prior, and no evidence-accumulation criterion that prevents immediate re-querying. This is the single most consequential failure mode.

5.2 Prose-without-coordinates. Questions L3_1 and L3_3 produced correct *room-level* answers in prose ("the entryway," "the living room") but no (x, y, z) coordinate in the answer payload. Under the two-path rubric of §3.5 these can still pass via the visual path (the retrieved frames support the prose answer), but they fail the coordinate path because no coordinate was ever emitted. This is a framework failure, not a VLM failure: the VLM "knew" the answer; the answer-formulation path lost the coordinate. A coordinate-extraction fallback (emit the top-1 retrieved frame's pose) would salvage every observed case.

5.3 Correct frames, offset coordinates. The retrieved frames are right but the reported coordinate drifts from ground truth by 2–5 meters. Consistent with a metadata-extraction bug in the answer-formulation path, not a reasoning failure.

5.4 Texture–surface confusion (embedder-side). For L5_3 ("flat surface that is NOT a floor"), the model correctly read the marble texture in `frame_000343` but mislabeled the surface as a "marble countertop" when it was a marble *floor*. The retrieval is correct in the sense of returning a visually relevant frame; the failure is in the *interpretation* of surface geometry. Because contrastive image–text encoders are trained against captions that

describe *what is in* a frame rather than *which surface is which*, two frames containing the same material—a marble floor and a marble countertop—live in nearly the same embedding neighborhood. The encoder cannot, in general, disambiguate them.

The failure is therefore embedder-side in origin but *not exclusively embedder-side in remedy*: the agent layer can in principle close it without changing the retrieval substrate. A follow-up VLM call, conditioned on the candidate frame and asking explicitly "is the marble surface in this frame a floor or a countertop?", would resolve the case in a single bounded inference because the open VLMs evaluated here *can* read horizontal versus vertical surface orientation from the image when prompted to do so. The structural fix is the same as the cheap-verifier mechanism proposed in §7 for query routing: a verification step that consults the visual content of the candidate, not just its embedding similarity, before the answer is emitted. The embedder ambiguity is real, but it is not a hard ceiling.

5.5 Rare multi-tool strategy. Only 1 of 19 queries triggered a non-text retriever (L4_3, the agent-start question; quantified in §4.6). The time and position retrievers are systematically under-used, implying that the system prompt or the tool descriptions do not sufficiently prime the VLM to reach for them when text retrieval fails. The fact that the model *can* call them when triggered means the lever is the prompt, not the model.

5.6 Type-classifier mis-routing (cross-model). RAVEN's first reasoning step asks the VLM to classify the question into one of {position, text, time, binary}; a position-typed answer triggers the coordinate-emission path, while a text-typed answer skips it entirely. Qwen3-VL on L3_1 and L3_3 classified both questions as `type=text` ("the question is asking about which room is closest to the entry, which is a descriptive question about location"), and then ran a perfectly reasonable text retrieval that landed on the right frames (`frame_000044` for L3_1, the living-room frames at `[8.149,-3.812,0.44]` for L3_3) but discarded the position because it had already committed to a text-only response. This is a different failure pathway from §5.2: there, the classifier picks `position` and the coordinate is lost downstream; here, the classifier itself mis-routes the question and the rest of the pipeline correctly follows from a wrong premise. The lever is the same as §5.5—the type-classifier prompt—but the symptom is sharper because no amount of retrieval-loop tuning can recover a coordinate the agent decided not to emit.

6. Discussion

6.1 Why the embedder-only floor is high on L1–L2

The RAVEN paper's negative result—medium open VLMs ≈ embedder-only—reproduces on the DARPA simulation data for L1 direct queries. The mechanism is now visible: QQMM's top-1 retrieval for a keyword query already lands on the correct frame, so a VLM that simply parrots the retrieved frame's location is no better than retrieval-only. **The LLM adds value precisely at L3–L5**, where retrieval alone is insufficient: spatial relationships between rooms, multi-step composition involving time, and explicit negation. This is exactly the per-level pattern visible in Figure 4. This has a direct deployment implication: if the workload is dominated by L1-style direct lookups, the LLM is overhead. If the workload is dominated by L3–L5, it is essential.

6.2 What scales with VLM size

Across the four open backbones (Gemma 3 27B, Gemma 4 31B, Qwen2.5-VL 32B, Qwen3-VL 32B), retrieval-level scores cluster within a 3-point band: **12, 13, 13, 15 out of 19**. There is no monotonic scaling trend. Per Table 1, all four backbones sit in a tight 16–22 GB VRAM range; the smallest of the four (Gemma 3 27B at ≈16 GB) lands at 13/19, while the highest-scoring backbone (Gemma 4 31B at ≈20 GB) is two questions ahead at 15/19. Memory footprint and accuracy do not correlate.

Scaling the VLM up does not buy retrieval-level accuracy on this suite, and the headline §4.2 result holds: **the bottleneck is downstream of retrieval**. When an agent fails to produce a usable coordinate, it has typically retrieved the right frame but emitted prose without a coordinate, or extracted the wrong pose from the right frame (§5.2, §5.3). A larger VLM is asked to do its job more often only because retrieval succeeds more often, but the framework-level coordinate-emission step has nothing to do with model size.

The lever for offline open-VLM RAVEN is therefore not the VLM. It is the answer-formulation layer (§6.3).

6.3 Actionable framework recommendations

Three changes are tractable at the agent/framework layer, none of which require fine-tuning the VLM and none of which touch the retrieval substrate:

- **Query diversification.** After K consecutive identical retrievals (say $K = 2$), force a prompt-level rewrite. Breaks the retrieval-loop failure mode (§5.1).

- **Coordinate fallback.** When the VLM returns a textual room name with no coordinate, emit the top-1 retrieved frame's pose. Salvages every observed prose-without-coordinates case (§5.2).
- **Tool-use priming.** Add few-shot exemplars in `agent_system_prompt.txt` specifically showing the time and position retrievers being used after an initial text retrieval fails. Addresses §5.5.

None of these three fixes is ablated in this work; their measured lift is left to follow-up. Coordinate fallback is the cheapest (approximately a one-line change in the answer-formatter) and is the obvious next experiment.

6.4 Limitations

- **Single scene, 843 frames.** The evaluation is on one trajectory; the RAVEN paper's scale claims (up to ∼3,000 frames on FindingDory long) are not probed here.
- **Embedder vs. agent: not a like-for-like comparison.** The headline embedder-only baseline uses top-1 retrieval, while the agent has access to top-K (K=5) and can reformulate the query within the loop (§4.4). The +8-question lift therefore bounds the agent's marginal contribution from above; an embedder + single-shot reformulation baseline (not run in this work) would isolate the VLM's reasoning contribution more cleanly.
- **Variance not fully quantified.** I report qualitative observations across runs but do not report per-model standard deviations. The RAVEN paper reports ±3–6% for open VLMs; similar variance is expected here.
- **Small-n uncertainty.** With $n = 19$ questions overall and $n = 3$–4 per level, single-question swings move the headline number by roughly 5 percentage points and a per-level percentage by 25 points. Adjacent embedder rows in Table 3 that differ by one question (CLIP ViT-L 5/19 vs. ViT-H 6/19) or that tie outright (ViT-H 6/19, SigLIP 6/19) cannot be statistically distinguished at this sample size; the L3–L5 lift pattern, however, spans ≥ 8 questions across three levels and is robust to single-question swings. Per-level cells should be read as point estimates rather than precise rates.
- **Sim-to-real gap.** All results are on Habitat-rendered frames; the real-robot results in the RAVEN paper are not replicated here.

- **Replication, not novel method.** The retrieval substrate, the tool-use loop, and the embedder-only baseline are established. Contribution is open-VLM coverage on new evaluation data and the failure-mode diagnosis.

6.5 Key takeaways

- **The lift is the agent, not the retriever.** Across a 4.7× range of embedding dimensions (768 to 3584), retrieval-level accuracy varies by 11 percentage points; adding the VLM agent on the same retriever lifts it 42 percentage points.
- **The lift is concentrated on L3–L5.** L1 direct lookup is solved by retrieval alone; the agent earns its cost on spatial, multi-step, and negation queries where retrieval is structurally insufficient.
- **The remaining loss is agent-side and tractable.** Three framework-level fixes (query diversification, coordinate fallback, tool-use priming) target the dominant failure modes diagnosed here without touching the retrieval substrate or fine-tuning the VLM.

7. Future Work

Routing fast queries around the VLM agent. The per-level breakdown of §4.3 shows embedder-only retrieval is competitive on L1 and adequate on L2, while L3–L5 is where the VLM agent earns its cost. The natural next step is to *route* at inference time, but the design choice is non-obvious. Retrieval similarity is not a reliable proxy for answer correctness; the embedder will confidently retrieve visually similar but semantically wrong frames whenever rooms share furniture or lighting. A similarity-margin gate alone will short-circuit to wrong answers in exactly the cases it should defer. Three candidate gate mechanisms:

- *Cheap VLM verifier (draft + verify).* Embedder retrieves a top-1 candidate; a single bounded VLM call decides whether it answers the question. Accepted candidates return immediately; rejected ones escalate to the full agent loop. Costs one cheap inference per easy query versus the four-to-six calls of the full loop, and the verifier *sees* the candidate so confidently-wrong retrievals are caught.
- *Multi-signal triangulation.* Combine retrieval margin, a question-shape heuristic, and an answer-shape parse. All three must agree before the gate accepts. Preserves a true "no VLM call" fast path but is vulnerable to a wrong frame from a similar-looking room passing all three checks.

- *Learned classifier on the L1–L5 taxonomy.* Stretch goal; would require a substantially larger labeled set than nineteen questions.

The cheap-verifier path is the most defensible because it directly addresses the failure mode that dooms similarity-only gates.

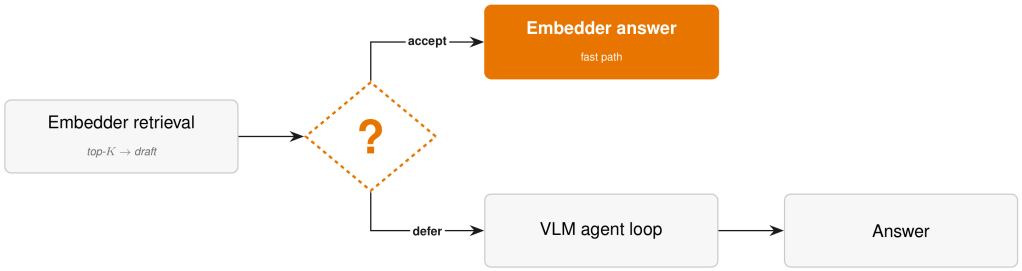

Figure 9. Proposed routing gate (future work, not in the system evaluated in this thesis). The embedder retrieves a top-K draft; a gate decides *accept* (return the embedder answer as a fast path) or *defer* (escalate into the full VLM agent loop). The cheap-verifier mechanism described above instantiates the gate with a single bounded VLM call.

Joint integration with full RAVEN. The gate, whichever mechanism, should be added to the full RAVEN system *jointly* with the §6.3 recommendations (query diversification, coordinate fallback, tool-use priming) rather than as disjoint patches. The changes interact: coordinate fallback shortens the agent path the gate is trying to avoid; query diversification reduces retrieval-loop failures that would cause the verifier to reject good candidates; tool-use priming changes which queries are reasoning-required.

Variance and scale. Replicate on multiple TIAMAT trajectories and on a FindingDory-sized memory (∼3,000 frames) with per-model standard deviations.

Training-side complement. Although the framework-level fixes do not require retraining, a separate line of work would adapt a smaller open VLM specifically to the retrieval-loop failure mode: instruction-tuning on synthetic transcripts that include forced query diversification and tool-switching.

8. Conclusion

This independent work extended the RAVEN evaluation to locally-served open-source VLMs over DARPA TIAMAT simulation data, authored a difficulty-stratified nineteen-question

suite specifically designed to separate retrieval-solvable from reasoning-required queries, and diagnosed open-VLM failure modes as predominantly *agent-side* (retrieval loops, coordinate-plumbing gaps, an under-used non-text retriever, and a question-type classifier that occasionally mis-routes positional questions into the prose path) rather than retrieval-substrate or base-VLM failures.

The headline quantitative result is that adding a VLM agent on top of QQMM-embed-v2 lifts retrieval-level accuracy from 7/19 (embedder-only) to 15/19 with Gemma 4 31B, with the entire gain concentrated on L3–L5 (spatial, multi-step, and negation reasoning). Increasing the embedder dimension by 4.7× (from 768 to 3584) closes only 11 percentage points of the gap, while adding the agent closes 42 percentage points. The lift is downstream of the retriever, not in it.

The practical implication for offline, air-gapped deployments of RAVEN on open models is direct: most of the gap to closed-VLM performance is recoverable without retraining, via three framework-level changes (query diversification, coordinate fallback, tool-use priming) that target the dominant failure modes identified here. The embedder-only pipeline remains a competitive fast path for L1 queries (consistent with RAVEN's own §5.3 recommendation), and the natural next step is to route between fast path and full agent at inference time using a cheap VLM verifier rather than a similarity-only gate.

Code and data availability

All evaluation question files (`test_questions_smoke.md`, `test_questions_smoke_qa.json`), per-model YAML configs in `cfgs/vlms/`, the modified evaluation harness, and full trace logs for the headline Gemma 4 31B run are maintained inside the **PRISM** project's private GitHub repository, available to advisers and committee members on request.

References

[1] Y. Hu, Z. Zheng, L. Zha, C. Xing, R. Singh, O. Hossain, A. Loquercio, and D. Shah. *RAVEN: Long-Horizon Reasoning and Navigation with a Visuo-Spatial-Temporal Memory.* Robotics: Science and Systems (RSS), 2026.

[2] A. Anwar, J. Welsh, J. Biswas, S. Pouya, and Y. Chang. *ReMEmbR: Building and Reasoning Over Long-Horizon Spatio-Temporal Memory for Robot Navigation.* IEEE International Conference on Robotics and Automation (ICRA), 2025.

[3] K. Yadav, S. K. Ramakrishnan, J. Turner, A. Gokaslan, et al. *FindingDory: A Benchmark for Memory-Based Navigation.* 2024.

[4] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger, and I. Sutskever. *Learning Transferable Visual Models from Natural Language Supervision.* International Conference on Machine Learning (ICML), 2021.

[5] X. Zhai, B. Mustafa, A. Kolesnikov, and L. Beyer. *Sigmoid Loss for Language-Image Pre-training.* International Conference on Computer Vision (ICCV), 2023.

[6] Y. Xue, D. Li, and G. Liu. *QQMM-embed-v2: A Multimodal Memory Embedding Model for Visuo-Spatial-Temporal Retrieval.* Hugging Face: `youzexue/QQMM-embed-v2`, 2025.

[7] J. Johnson, M. Douze, and H. Jégou. *Billion-Scale Similarity Search with GPUs.* IEEE Transactions on Big Data, 2019.

[8] M. Savva, A. Kadian, O. Maksymets, et al. *Habitat: A Platform for Embodied AI Research.* International Conference on Computer Vision (ICCV), 2019.

[9] Gemma Team, Google DeepMind. *Gemma 3: Open Multimodal Foundation Models.* Technical Report, 2024.

[10] Qwen Team, Alibaba. *Qwen2.5-VL Technical Report.* 2024.

[11] DARPA TIAMAT Competition documentation. Internal documentation, 2025.

A. Full Evaluation Question List

The full nineteen-question suite, with capability tags and ground-truth positions, is maintained in `test_questions_smoke.md` (human-readable) and `test_questions_smoke_qa.json` (machine-readable). Question summaries by level:

L1—Direct object recognition.

- L1_1: Where is the purple chair?
- L1_2: Where is the toilet?
- L1_3: Where is the white sofa?
- L1_4: Where are the kitchen cabinets?

L2—Indirect / attribute-based.

- L2_1: Where can I sit down comfortably?
- L2_2: Where can I wash my hands?
- L2_3: Where is something I could use to watch the news?

L3—Spatial reasoning.

- L3_1: What room is closest to the entry?
- L3_2: Where would I go to get from the kitchen to the bathroom?
- L3_3: Which area has the most furniture?
- L3_4: Where is the seating area that faces the TV?

L4—Multi-step inference.

- L4_1: I need to make coffee and sit down to drink it. Where should I go first?
- L4_2: If I spill water on the kitchen floor, where would I find something to clean it up?
- L4_3: Where did the agent start its exploration?
- L4_4: If I'm at the toilet and hear the TV, which direction would I walk?

L5—Negation / disambiguation.

- L5_1: Find a chair that is NOT in the kitchen.
- L5_2: There are multiple seating options. Find the one closest to the bathroom.
- L5_3: Find a flat surface that is NOT a floor.
- L5_4: Where did the agent spend the most time during its exploration?

B. Per-Model Output Directories

All runs are reproducible from `outputs/smoke_v2/raven_results/20q/`:

- `gemma4_31b_qqmm/`—Gemma 4 31B with QQMM-embed-v2. **Primary result.** Findings summary in `GEMMA4_31B_FINDINGS.md`.
- `gemma27b_qqmm/`—Gemma 3 27B with QQMM-embed-v2.
- `qwen25vl32b_qqmm/`—Qwen2.5-VL 32B (q4_K_M) with QQMM-embed-v2.
- `qwen3vl32b_qqmm/`—Qwen3-VL 32B instruct (q4_K_M) with QQMM-embed-v2.

Embedder-only baselines: `outputs/custom_eval/` with summary `EVAL_REPORT.md`.

C. Reproduction Commands

The headline result (Gemma 4 31B + QQMM, 15/19) reproduces with:

```
python remembr_static_eval_vlm.py \
  --vlm_config cfgs/vlms/gemma4-31b.yaml \
  --embedder_config cfgs/embedders/qqmm.yaml \
  --agent_config cfgs/agents/raven.yaml \
  --input_folder ./extracted_videos/smoke_full \
  --qa_file ./test_questions_smoke_qa.json \
  --caption_file ./extracted_videos/smoke_full/frames.json \
  --out_dir ./outputs/smoke_v2/raven_results/20q/gemma4_31b_qqmm \
  --memory_backend faiss \
  --top_k 5 \
  --device cuda
```

Swap `-vlm_config` to any of the configs in `cfgs/vlms/` to reproduce the per-model rows in §4.2. Embedder-only baselines reproduce with `run_custom_eval.py`, which loads the same FAISS index and applies top-1 retrieval without invoking a VLM.

D. Engineering Standards Used

This appendix documents the engineering standards, frameworks, and accepted industrial conventions used in the implementation.

- **PyTorch ($\geq 2.x$), CUDA 12.x.** Standard deep-learning frameworks for tensor computation and GPU acceleration. Used for embedder forward passes.
- **FAISS (Facebook AI Similarity Search), v1.7+.** Standard library for billion-scale similarity search. Cosine distance index over 3584-dim QQMM embeddings.
- **Hugging Face Transformers and Hub.** Standard model distribution and weight loading.
- **Ollama (≥ 0.3).** Local-serving framework for open-source LLMs/VLMs. Used to host all four VLM backbones.
- **YAML configuration format.** Standard human-readable config representation; per-model YAML configs in `cfgs/vlms/`.
- **Habitat-Sim 3D simulator.** Established research-community standard for embodied-AI simulation.

- **JSON for evaluation I/O.** Standard structured data interchange.
- **LaTeX / Markdown.** Standard scientific document formats.
- **Git for version control.**

The retrieval, tool-use, and prompt logic conform to RAVEN [1]'s reference implementation. No retrieval-substrate, tool, or prompt modifications are introduced beyond per-model YAML config additions.

E. Failure-Mode Cross-Tabulation

For each of the six failure modes identified in §5, the table below lists which questions exhibited the failure in the Gemma 4 31B + QQMM run. This is the quantitative footprint of the failure-mode taxonomy.

Failure mode	Affected questions	Count
5.1 Retrieval loops ($\geq$ 15 tool calls)	L3_1, L4_1, L5_4	3
5.2 Prose without coordinates	L3_1, L3_3	2
5.3 Correct frames, offset coordinates	six L4–L5 questions where retrieval was right but coordinate was off	6
5.4 Texture–surface confusion	L5_3	1
5.5 Rare multi-tool strategy	all questions *except* L4_3	18
5.6 Type-classifier mis-routing (Qwen3-VL)	L3_1, L3_3	2

L3_1 appears in two failure modes (retrieval loop *and* prose-without-coordinates), illustrating that the modes are not mutually exclusive.

F. Reproducibility Checklist

- **Hardware.** 2× NVIDIA L40 (48 GB VRAM each), Princeton ECE neuronic cluster.
- **Operating system.** Ubuntu Linux (kernel and exact release: see cluster image at run time).
- **CUDA / driver.** CUDA 12.x; driver version recorded in run-time logs.
- **PyTorch.** $\geq$2.x with CUDA 12 wheels.
- **Ollama.** $\geq$0.3 (exact build hash captured in YAML config metadata).

- **Model digests.** Open VLM backbones are served via Ollama as `gemma3:27b`, `gemma4:31b`, `qwen2.5vl:32b-q4_K_M`, and `qwen3-vl:32b-instruct-q4_K_M`; QQMM-embed-v2 is pulled from `youzexue/QQMM-embed-v2`. See `cfgs/vlms/` and `cfgs/embedders/` for the full YAML.
- **FAISS.** v1.7+; cosine-distance index over 3584-dim QQMM embeddings of 843 frames.
- **Top-K.** Fixed at 5 throughout.
- **Random seed.** Default (no explicit seed set in run `_0`).
- **Trajectory data.** `smoke_v2`, 843 egocentric Habitat frames; pose CSV `position_data_habitat_sm` included in the release.
- **Question file.** `test_questions_smoke_qa.json` (19 graded questions).
- **Git revision.** Commit hash of fork at run time: `d15ab27`.